as virtualisation in providing isolation and resource management, they still have a clear edge in terms of being a lightweight execution environment. Containers are isolated at the process level and share the kernel of the host machine, which means that a container doesn't include a complete operating system, leading to faster start-up times.

- *Layering*: Containers, being ultra-lightweight in the operating environment, function in layers and every layer performs individual tasks, leading to minimal disk utilisation for images.

With the general adoption of cloud computing platforms, integrating and monitoring container based technologies is of utmost necessity. Container management and automation tools are regarded as important areas for development as today's IT organisations need to manage and monitor distributed and cloud/native based applications.

Containers can be visualised as the future of virtualisation, and strong adoption of containers can already be seen in cloud and Web servers. So, it is very important for systems administrators to be aware of various container management tools that are also open source. Let's take a look at what I consider are the best tools in this domain.

Apache Aurora

Apache Aurora is a service scheduler that runs on top of Apache Mesos, enabling users to run long-running services and to take appropriate advantage of Mesos' scalability, fault tolerance and resource isolation. Aurora runs applications and services across a shared pool of host machines and is responsible for keeping them up and running 24x7. In case of any technical failures, Aurora intelligently does the task of rescheduling over other running machines.

Components

- *Scheduler:* This is regarded as the primary interface for the user to work on the cluster and it performs various tasks like running jobs and managing Mesos.
- *Client:* This is a command line tool that exposes primitives, enabling the user to interact with the scheduler. It also contains *Admin_Client* to run admin commands especially for cluster administrators.
- *Executor:* This is responsible for carrying out workloads, executing user processes, performing task checks and registering tasks in Zookeeper for dynamic service discovery.
- *Observer:* This provides browser based access to individual tasks running on worker machines and gives detailed analysis of processes being executed. It enables the user to browse all the sandbox task directories.
- *Zookeeper:* It performs the task of service discovery.
- *Mesos Master:* This tracks all worker machines and ensures resource accountability. It acts as the central node that controls the entire cluster.

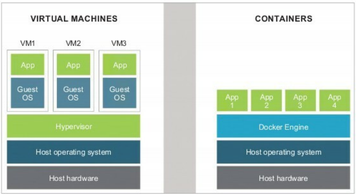

Figure 1: Differences between the architecture of virtual machines and containers

- *Mesos Agent:* This receives tasks from the scheduler and executes them. It basically interfaces with Linux isolation groups like cgroups, namespace and Docker to manage resource consumption.

Features

- Scheduling and deployment of jobs.
- Resource quota and multi-user support.
- Resource isolation, multi-tenancy, service discovery and Cron jobs.

Official website: *http://aurora.apache.org/*
Latest version: 0.18.0

Apache Mesos

Apache Mesos is an open source cluster manager developed by the University of California, Berkeley, and provides efficient resource isolation and sharing across distributed applications or frameworks. Apache Mesos abstracts the CPU, memory, storage and other computational resources away from physical or virtual machines and performs the tasks of fault tolerance.

Apache Mesos is being developed on the same lines as the Linux kernel. The Mesos kernel is compatible with every machine and provides a platform for various applications like Apache Hadoop, Spark, Kafka and Elasticsearch with APIs for effective resource management, and scheduling across data centres and cloud computing functional environments.

Components

- *Master:* This enables fine-grained resource sharing across frameworks by making resource offers.
- *Scheduler:* Registers with the Master to be offered resources.
- *Executor:* It is launched on agent nodes to run the framework tasks.

Features

- Isolation of computing resources like the CPU, memory and I/O in the entire cluster.
- Supports thousands of nodes, thereby providing linear scalability.